State Merging Algorithms Need Short Strings, Neural Networks Need Long Ones (SP: super tentative title pls feel free to change)

Anonymous ACL submission

1 Introduction

(LS: If we have time, we could run RPNI and EDSM separately also. But for now we just have results from our single gridsearch winner (RPNI-like)) This paper compares the results of a state merging algorithm and four neural network architectures on an extension of MLRegTest, a benchmark for the learning of regular languages (van der Poel et al., 2024). The training data of MLRegTest contains only strings of at least length 20, but Soubki and Heinz (2023) demonstrated that augmenting this with shorter strings led to significant improvement in the performance of the state merging algorithms. Following Soubki and Heinz, we compare performance of both state merging and neural models when trained on the original MLRegTest data, the augmented Soubki and Heinz data, and training data containing *only* strings of length 19 or less. (TODO: We find that neural models outperform state merging on the original and augmented datasets, but that the state merging algorithms are less hindered by the absence of long strings, outperforming most neural archetectures on the short string only data.)

2 Learning Systems

2.1 State-Merging Systems

Following Soubki and Heinz (2023), we make use of the state-merging algorithms provided by FlexFringe (Verwer and Hammerschmidt, 2017, 2022).[1] FlexFringe provides implementations of the EDSM (Lang et al., 1998) and RPNI (Oncina et al., 1992)state-merging algorithms:[2] both take as input a set of strings labeled by their membership in the target langauge, and construct a prefix tree from these strings. A search is then conducted for pairs of states to merge: RPNI does so in a breadth-first fashion, selecting pairs closest to the initial state. EDSM instead computes a score for all possible pairs and attempts to merge the one with the highest score.[3]

FlexFringe also provides several other parameters which impact learning behavior. To find the best model, we conducted a limited gridsearch over the following subset[4]:

- PerformFirst: whether the algorithm should perform EDSM or RPNI
- Extend: extend (color red) blue states that cannot be merged with any red
- ShallowFirst: consider coloring blue states closest to the root first
- SearchStrategy: searchdeep, searchlocal, searchglobal, searchpartial, none (TODO: They have absolutely no documentation of what these parameters do, should we just guess?)

Grid searching was carried out on the small partition of the PlusShort data using overall accuracy on the development set (see §3). The best performing model, used in the remainder of the paper, was PerformFirst = 0 (i.e., RPNI), Extend = 1, ShallowFirst = 0, and SearchStrategy = SearchDeep.

2.2 Neural Systems

Following van der Poel et al. (2024), we test four neural architectures, all consisting of a trainable

[1] We use the same version of FlexFringe used by Soubki and Heinz: commit `4d1449c` of the software available here.

[2] We exclude ALERGIA from consideration because of the infeasibly high compute demands and comparatively low performance noted by Soubki and Heinz

[3] Notably, the implementation of RPNI in FlexFringe differs slightly from that in Oncina et al. (1992): while Oncina et al. merged states in length-lexicographic order when ties arise, FlexFringe instead calculates a score, akin to EDSM.

[4] We held several other parameters at their default state because modifying them led to errors in the FlexFringe executable. These are: ReverseTraces, DepthCheck, SymbolCheck, SinksOn, SinkCount, and MergeScore.

FACTOR	POSSIBLE LEVELS
AlphabetSize	4, 16
Tier	3, 4, 7, 16
LanguageClass	SL, coSL, TSL, TcoSL, SP, coSP, LT, TLT, PT, LTT, TLTT, PLT, TPLT, SF, Zp, Reg
FactorWidth	0, 2, 3, 4, 5, 6
ThresholdSize	0, 1, 2, 3, 5
Index	0, 1, 2, 3, 4, 5, 6, 7, 8, 9
TrainSize	Small (1k), Mid (10k), Large (100k)
DataType	ONLYSHORT, PLUSSHORT ONLYLONG
Model	FLEXFRINGE, SIMPLE GRU, LSTM, TRANSFORMER

Table 1: The values used in our experimental setup.

MODEL	OS	PLUSSHORT			ONLYLONG		
		S	M	L	S	M	L
FF	0.709	0.747	0.775	X	0.534	0.560	X
Simple	0.679	0.732	0.780	0.834	0.720	0.774	0.832
GRU	**0.792**	**0.907**	**0.957**	0.963	**0.847**	**0.942**	0.949
LSTM	0.646	0.727	0.868	0.924	0.638	0.837	0.912
Trans.	0.709	0.762	0.809	0.856	0.754	0.806	0.806

Table 2: F1 score by model, averaged across languages and test types. (SP: this might be too wide)

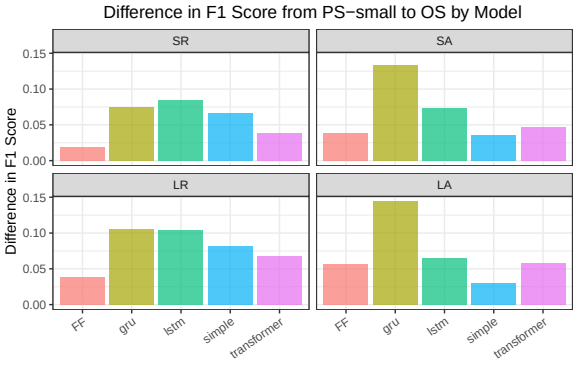

Figure 1: Difference in F1 between PLUSSHORT (small training) and ONLYSHORT by model & test type.

embedding layer followed by a unidirectional recurrent module (either a **SIMPLE** RNN, a **GRU**, or an **LSTM**) or two consecutive **TRANSFORMER** blocks, dense feed-forward layers, dropout, layer normalization, and softmax activation. All neural networks are implemented with TensorFlow and Keras (Abadi et al., 2015; Chollet, 2015) and use the same hyperparameters as van der Poel et al. (2024).[5]

3 Evaluation

We evaluate our learning models with an extension of MLREGTEST (van der Poel et al., 2024), which contains training, development, and test data with and equal number of positive and negative examples of strings from 1,800 languages. We follow Soubki and Heinz (2023) in excluding those languages with alphabet sizes of 64 due to unicode errors, thus considering 1,140 languages.

The original MLREGTEST training data contains strings with length 20-29. Soubki and Heinz conducted a follow-up experiment in which this was augemented with roughly 1,000 short training strings (length 1-19). The authors found that this change significantly improved the performance of FLEXFRINGE models. In the current work, we build on Soubki and Heinz by considering three training setups:

ONLYSHORT: models are trained on only the short strings from Soubki and Heinz (2023)

PLUSSHORT: models are trained on MLREGTEST augmented with short strings, replicating Soubki and Heinz (2023)

ONLYLONG: models are trained on the original MLREGTEST data

MLREGTEST is designed to investigate a number of potential factors which may contribute to model performance, summarized with our additions in Table 1. It additionally contains four testing sets per language, based on the length of the strings (SHORT = 20-29, LONG = 31-50) and the method of generation (RANDOM, sampled randomly without replacement or ADVERSARIAL, with paired positive and negative examples with an edit distance of 1). We report results separately on each test set.

4 Results

Table 2 gives the F1 score of each model, averaged across languages and test types; breakdowns by test type are given in Tables 3-5 in the Appendix. Though the GRU consistently outperforms the other models, we note that it takes much greater of a hit in performance between PLUSSHORT and ONLYSHORT than FLEXFRINGE, which outperforms all other neural architectures on ONLYSHORT. Figure 1 illustrates the difference in average F1 between the PLUSSHORT small train size and ONLYSHORT by model and test set: FLEXFRINGE has the smallest difference in performance on the random test sets, and is comparable to the SIMPLE RNN on the adversarial ones; the

[5]Implementations of the neural networks are available at: https://github.com/heinz-jeffrey/subregular-learning/

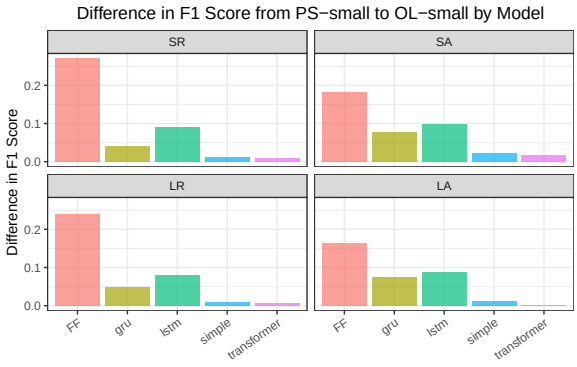

Figure 2: Difference in F1 between PLUSSHORT and ONLYLONG (small size) by model & test type.

GRU shows the largest difference in performance on these test sets.

Similarly, we note that FLEXFRINGE sees the largest jump in performance from ONLYLONG to PLUSSHORT; this is illustrated in Figure 2. Taken together, these results suggest that FLEXFRINGE benefits far more from the inclusion of short strings, while the neural models — particularly the GRU — rely heavily on long strings to achieve their performance. Further breakdowns of performance by language class are given in Figures 3-5 in the Appendix.

5 Discussion

(TODO: key points:

- GRU seems to be strongest overall
- FF gets more benefit from short strings than neural models do
- FF generally is more competitive on the small training data
- natural language = short + small

)

6 Conclusions

References

Martín Abadi, Ashish Agarwal, Paul Barham, Eugene Brevdo, Zhifeng Chen, Craig Citro, Greg S Corrado, Andy Davis, Jeffrey Dean, Matthieu Devin, and 1 others. 2015. Tensorflow: Large-scale machine learning on heterogeneous systems.

François Chollet. 2015. Keras. https://github.com/fchollet/keras.

Kevin J Lang, Barak A Pearlmutter, and Rodney A Price. 1998. Results of the abbadingo one dfa learning competition and a new evidence-driven state merging algorithm. In *International Colloquium on Grammatical Inference*, pages 1–12. Springer.

José Oncina, Pedro Garcia, and 1 others. 1992. Inferring regular languages in polynomial update time. *Pattern recognition and image analysis*, 1(49-61):10–1142.

Adil Soubki and Jeffrey Heinz. 2023. Benchmarking state-merging algorithms for learning regular languages. In *Proceedings of 16th edition of the International Conference on Grammatical Inference*, volume 217 of *Proceedings of Machine Learning Research*, pages 181–198. PMLR.

Sam van der Poel, Dakotah Lambert, Kalina Kostyszyn, Tiantian Gao, Rahul Verma, Derek Andersen, Joanne Chau, Emily Peterson, Cody St. Clair, Paul Fodor, Chihiro Shibata, and Jeffrey Heinz. 2024. Mlregtest: A benchmark for the machine learning of regular languages. *Journal of Machine Learning Research*, 25(283):1–45.

Sicco Verwer and Christian Hammerschmidt. 2022. FlexFringe: Modeling software behavior by learning probabilistic automata. *arXiv preprint*.

Sicco Verwer and Christian A. Hammerschmidt. 2017. FlexFringe: A passive automaton learning package. In *2017 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, pages 638–642.

A Appendix

(TODO: add graphs?) (SP: my pref would be to have one or two punchy graphs in the main text and lots of numbers in the appendix)

MODEL	SR	SA	LR	LA
FF	0.808	0.662	0.746	0.620
Simple	0.826	0.587	0.738	0.566
GRU	0.903	0.713	0.861	0.692
LSTM	0.795	0.545	0.732	0.510
Trans.	0.867	0.629	0.778	0.562

Table 3: F1 by model & test type on ONLYSHORT

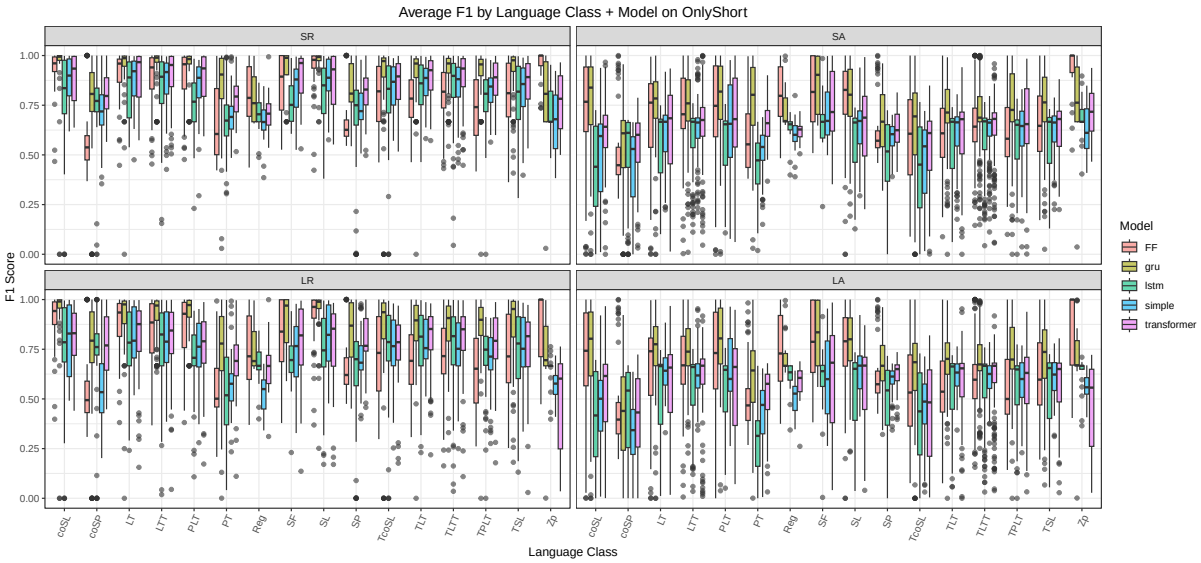

Figure 3: Average F1 by model, test set, and language class for ONLYSHORT

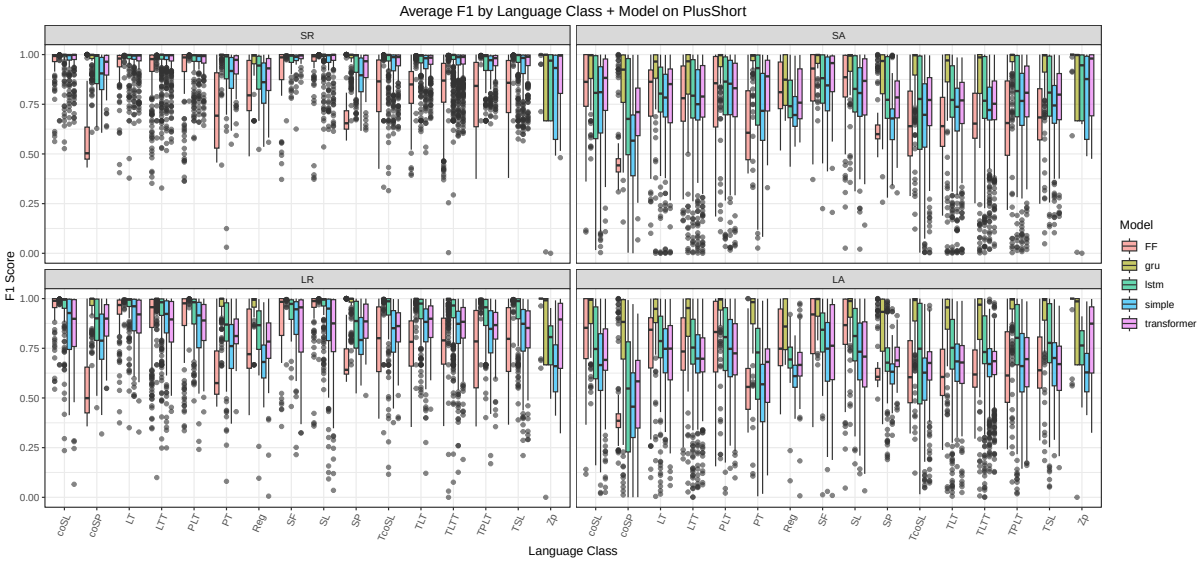

Figure 4: Average F1 by model, test set, and language class for PLUSSHORT, averaged across training sizes (TODO: re-make once Large FF is in)

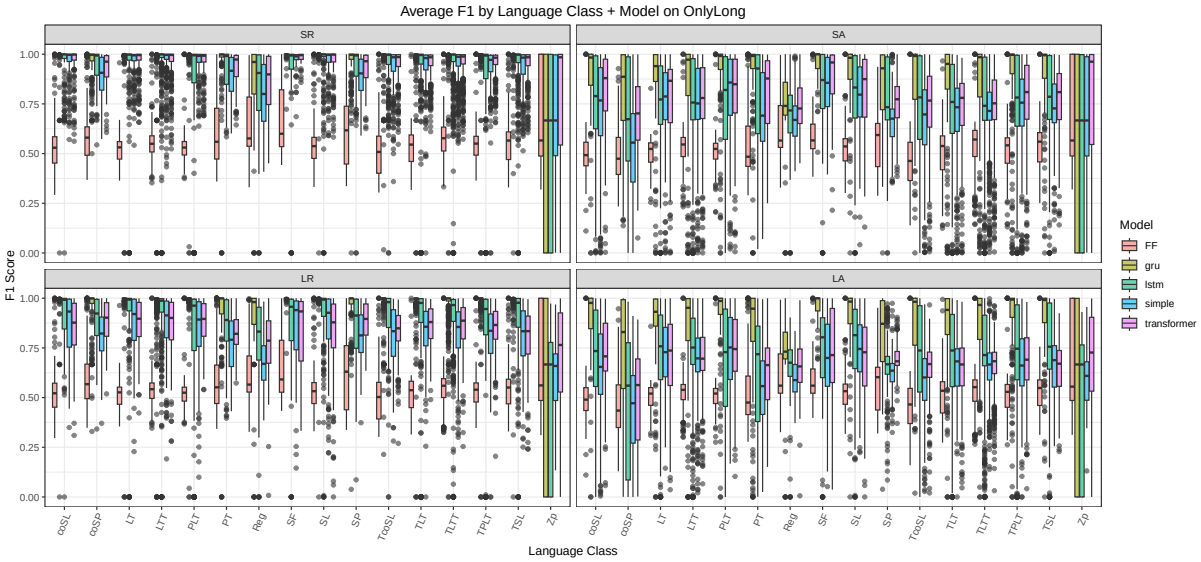

Figure 5: Average F1 by model, test set, and language class for ONLYLONG, averaged across training sizes (TODO: re-make once Large FF is in)

SMALL	SR	SA	LR	LA
FF	0.827	0.701	0.785	0.676
Simple	0.892	0.622	0.820	0.596
GRU	0.978	0.847	0.966	0.836
LSTM	0.880	0.618	0.836	0.575
Trans.	0.906	0.676	0.846	0.620
MID	SR	SA	LR	LA
FF	0.846	0.729	0.817	0.707
Simple	0.940	0.692	0.844	0.645
GRU	0.994	0.925	0.988	0.921
LSTM	0.972	0.808	0.942	0.751
Trans.	0.956	0.762	0.854	0.663
LARGE	SR	SA	LR	LA
FF	X	X	X	X
Simple	0.982	0.824	0.830	0.701
GRU	0.992	0.936	0.990	0.934
LSTM	0.997	0.902	0.953	0.844
Trans.	0.989	0.858	0.861	0.714

Table 4: F1 by model, test type, and train size on PLUSSHORT

SMALL	SR	SA	LR	LA
FF	0.557	0.519	0.545	0.513
Simple	0.880	0.601	0.812	0.585
GRU	0.937	0.769	0.918	0.762
LSTM	0.790	0.521	0.755	0.487
Trans.	0.898	0.659	0.840	0.619
MID	SR	SA	LR	LA
FF	0.585	0.544	0.571	0.538
Simple	0.936	0.690	0.830	0.640
GRU	0.981	0.908	0.976	0.905
LSTM	0.942	0.778	0.911	0.716
Trans.	0.952	0.761	0.848	0.662
LARGE	SR	SA	LR	LA
FF	X	X	X	X
Simple	0.978	0.822	0.826	0.700
GRU	0.976	0.925	0.974	0.922
LSTM	0.983	0.895	0.941	0.831
Trans.	0.987	0.859	0.860	0.716

Table 5: F1 by model, test type and trainsize on ONLYLONG